Web 3.0 Unveiled - Day 4

Mentored by FEC

Arrays, Strings

Array can have a compile-time fixed size or a dynamic size.

```
pragma solidity ^0.8.10;


contract Array {


   // Declare a string variable which is public
   string public greet = "Hello World!";
   // Several ways to initialize an array
   // Arrays initialized here are considered state variables that get stored on
the blockchain
   // These are called storage variables
   uint[] public arr;
   uint[] public arr2 = [1, 2, 3];
   // Fixed sized array, all elements initialize to 0
   uint[10] public myFixedSizeArr;
   /*
       Name of the function is get
       It gets the value of element stored in an array's index
   */
   function get(uint i) public view returns (uint) {
       return arr[i];
   }


   /*
    Solidity can return the entire array.
    This function gets called with and returns a uint[] memory.
    memory - the value is stored only in memory and not on the blockchain
             it only exists during the time the function is being executed.


    Memory variables and Storage variables can be thought of as similar to RAM
vs Hard Disk.
    Memory variables exist temporarily during function execution, whereas
Storage variables
```

```
    are persistent across function calls for the lifetime of the contract.
    Here the array is only needed for the duration while the function executes
and thus is declared as a memory variable
    */
    function getArr(uint[] memory _arr) public view returns (uint[] memory) {
        return _arr;
    }


    /*
        This function returns string memory.
        The reason memory keyword is added is because string internally works as
an array
        Here the string is only needed while the function executes.
    */
    function foo() public returns (string memory) {
        return "C";
    }


    function doStuff(uint i) public {
        // Append to array
        // This will increase the array length by 1.
        arr.push(i);
        // Remove last element from array
        // This will decrease the array length by 1
        arr.pop();
        // get the length of the array
        uint length = arr.length;
        // Delete does not change the array length.
        // It resets the value at index to it's default value,
        // in this case it resets the value at index 1 in arr2 to 0
        uint index = 1;
        delete arr2[index];
        // create array in memory, only fixed size can be created
        uint[] memory a = new uint[](5);
        // create string in memory
        string memory hi = "hi";
    }


}
```

Enums

The word Enum stands for Enumerable. They are user defined types that contain human readable names for a set of constants, called members. They are commonly used to restrict a variable to only have one of a few predefined values. Since they are just an abstraction for human readable constants, in actuality, they are internally represented as uints.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.10;

contract Enum {
    // Enum representing different possible shipping states
    enum Status {
        Pending,
        Shipped,
        Accepted,
        Rejected,
        Canceled
    }

    // Declare a variable of the type Status
    // This can only contain one of the predefined values
    Status public status;

    // Since enums are internally represented by uints
    // This function will always return a uint
    // Pending = 0
    // Shipped = 1
    // Accepted = 2
    // Rejected = 3
    // Canceled = 4
    // Value higher than 4 cannot be returned
    function get() public view returns (Status) {
        return status;
    }

    // Pass the desired Status enum value as a uint
    function set(Status _status) public {
        status = _status;
    }
```

```
    // Update status enum value to a specific enum member, in this case, to the
Canceled enum value
    function cancel() public {
        status = Status.Canceled; // Will set status = 4
    }
}
```

Structs

The concept of structs exists in many high level programming languages. They are used to define your own data types which group together related data.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.10;

contract TodoList {
    // Declare a struct which groups together two data types
    struct TodoItem {
        string text;
        bool completed;
    }

    // Create an array of TodoItem structs
    TodoItem[] public todos;

    function createTodo(string memory _text) public {
        // There are multiple ways to initialize structs

        // Method 1 - Call it like a function
        todos.push(TodoItem(_text, false));

        // Method 2 - Explicitly set its keys
        todos.push(TodoItem({ text: _text, completed: false }));

        // Method 3 - Initialize an empty struct, then set individual
properties
```

```
        TodoItem memory todo;
        todo.text = _text;
        todo.completed = false;
        todos.push(todo);
    }

    // Update a struct value
    function update(uint _index, string memory _text) public {
        todos[_index].text = _text;
    }

    // Update completed
    function toggleCompleted(uint _index) public {
        todos[_index].completed = !todos[_index].completed;
    }
}
```

View and Pure Functions

You might have noticed that some of the functions we have been writing specify one of either a `view` or `pure` keyword in the function header. These are special keywords which indicate specific behavior for the function.

Getter functions (those which return values) can be declared either `view` or `pure`.

- `View`: Functions which do not change any state values
- `Pure`: Functions which do not change any state values and also do not read any state values

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.10;



contract ViewAndPure {

    // Declare a state variable
```

```
    uint public x = 1;



    // Promise not to modify the state (but can read state)

    function addToX(uint y) public view returns (uint) {

        return x + y;

    }



    // Promise not to modify or read from state

    function add(uint i, uint j) public pure returns (uint) {

        return i + j;

    }

}
```

Resources for further learning

- Cryptozombies
- Solidity by Example
- Solidity docs